

Clean Architecture

Atividade

Questões

1. Problema: código Flutter sem arquitetura

1. Explique o que significa dizer que um projeto Flutter está “sem arquitetura”. Cite pelo menos três sinais de que o código está desorganizado.
2. Imagine uma tela de login em que o botão chama diretamente uma API, valida os campos, salva o token no armazenamento local e navega para a próxima tela. Quais problemas essa abordagem pode causar?
3. Analise o seguinte cenário: um app possui regras de negócio espalhadas em vários widgets. O que pode acontecer quando uma regra precisar ser alterada?
4. Explique por que misturar interface, regra de negócio e acesso a dados no mesmo arquivo dificulta manutenção, testes e reaproveitamento de código.
5. Dê um exemplo de um código ou situação em Flutter onde o excesso de responsabilidade dentro de um widget prejudica o projeto.
6. Pesquise e explique o conceito de “código espaguete”. Como esse problema pode aparecer em projetos Flutter?

2. Conceito de Clean Architecture

1. Explique, com suas palavras, o que é Clean Architecture e qual problema ela tenta resolver no desenvolvimento de software.
2. Quais são os principais objetivos da Clean Architecture em um projeto Flutter? Cite pelo menos três.
3. Explique por que a Clean Architecture busca separar regras de negócio, interface e detalhes externos, como banco de dados e APIs.
4. Dê um exemplo de uma funcionalidade simples, como login, cadastro ou listagem de produtos, e explique como ela poderia ser dividida usando Clean Architecture.
5. Clean Architecture é sempre necessária? Explique em quais tipos de projeto ela pode ser vantajosa e em quais pode gerar complexidade desnecessária.
6. Pesquise o conceito de “independência de frameworks”. Como esse princípio se aplica ao Flutter?

3. Regra da dependência

1. Explique o que é a regra da dependência na Clean Architecture.
2. Por que as camadas mais internas não devem depender das camadas mais externas?

3. Em um projeto Flutter, por que a camada domain não deve importar arquivos da camada presentation?
4. Analise o seguinte problema: uma entidade da camada domain importa `package:flutter/material.dart`. Qual princípio foi violado? Que consequência isso pode trazer?
5. Dê um exemplo correto de dependência entre camadas usando presentation, domain e data.
6. Explique como o uso de interfaces ou classes abstratas ajuda a respeitar a regra da dependência.

4. Camadas: presentation, domain e data

1. Explique a responsabilidade da camada presentation em um projeto Flutter.
2. Explique a responsabilidade da camada domain. Que tipos de arquivos normalmente ficam nessa camada?
3. Explique a responsabilidade da camada data. Por que ela costuma lidar com APIs, banco local e conversão de dados?
4. Questão-problema: uma tela Flutter está acessando diretamente uma API REST usando `http.get`. Em qual camada esse código deveria estar? Justifique.
5. Dê um exemplo de uma funcionalidade de “listar tarefas” separando o que ficaria em presentation, domain e data.
6. Quais problemas podem surgir quando a camada presentation conhece diretamente detalhes da camada data?

5. Entity vs Model

1. Explique a diferença entre Entity e Model em Clean Architecture.
2. Por que uma Entity deve representar uma regra ou conceito do domínio, e não necessariamente o formato exato vindo da API?
3. Dê um exemplo de uma entidade User e de um model UserModel. Explique a diferença entre os dois.
4. Questão-problema: uma API retorna um campo chamado `user_name`, mas no aplicativo o domínio trabalha com `name`. Em qual classe essa conversão deve acontecer? Justifique.
5. Explique por que não é uma boa prática fazer uma entidade depender diretamente de métodos como `fromJson()` e `toJson()`.
6. Em projetos pequenos, é aceitável usar apenas Model sem Entity? Explique vantagens e riscos dessa decisão.

6. Repository vs DataSource

1. Explique o que é um Repository e qual é seu papel na Clean Architecture.
2. Explique o que é um DataSource. Cite exemplos de fontes de dados em um app Flutter.
3. Qual a diferença entre Repository e DataSource? Explique usando um exemplo de login.

4. Questão-problema: um app busca produtos ora da API, ora do cache local. Como o Repository pode decidir qual DataSource utilizar?
5. Por que a camada domain normalmente conhece apenas a abstração do Repository, e não os DataSources concretos?
6. Dê um exemplo de erro comum envolvendo Repository e DataSource em projetos Flutter.

7. Use Cases

1. Explique o que é um Use Case na Clean Architecture.
2. Por que um Use Case deve representar uma ação específica do sistema, como LoginUser, GetProducts ou CreateTask?
3. Dê um exemplo de Use Case para autenticação de usuário. Explique quais dados ele recebe e o que retorna.
4. Questão-problema: uma tela de cadastro possui validações, chamadas de API e regras para definir se o usuário pode se cadastrar. Quais partes poderiam ser movidas para um Use Case?
5. Explique a diferença entre colocar uma regra de negócio em um Use Case e colocá-la diretamente em um Controller, Cubit, Bloc ou ViewModel.
6. Use Cases podem deixar o projeto com muitos arquivos. Quando essa separação vale a pena? Quando pode ser exagerada?

8. Gerenciamento de estado dentro da presentation

1. Explique o que é gerenciamento de estado em Flutter.
2. Por que o gerenciamento de estado pertence à camada presentation e não à camada domain?
3. Compare pelo menos duas abordagens de gerenciamento de estado, como setState, Provider, Riverpod, Bloc, Cubit ou ChangeNotifier.
4. Questão-problema: uma tela precisa mostrar loading, sucesso e erro ao buscar dados de uma API. Como você representaria esses estados na camada presentation?
5. Explique por que um Bloc, Cubit, Controller ou ViewModel não deve conter diretamente código de API ou SQL.
6. Dê um exemplo de fluxo: a tela chama um Controller, o Controller chama um Use Case, o Use Case chama um Repository. Explique o papel de cada parte.

9. Organização de pastas

1. Por que a organização de pastas é importante em projetos Flutter maiores?
2. Explique a diferença entre organizar pastas por camada e organizar pastas por funcionalidade.
3. Dê um exemplo de estrutura de pastas para um app de tarefas usando Clean Architecture.

4. Questão-problema: um projeto tem arquivos de login espalhados em várias pastas sem padrão. Quais dificuldades isso pode causar para manutenção?
5. Como a organização de pastas pode ajudar novos desenvolvedores a entenderem o projeto mais rapidamente?
6. Explique por que uma boa estrutura de pastas não resolve sozinha problemas de arquitetura.

10. Diferentes maneiras de organização de pastas

1. Pesquise e explique a organização por camadas, por exemplo:

```
lib/  
  presentation/  
  domain/  
  data/
```

Quais são suas vantagens e desvantagens?

2. Pesquise e explique a organização por features, por exemplo:

```
lib/  
  features/  
    login/  
    products/  
    profile/
```

Quais são suas vantagens e desvantagens?

3. Explique uma organização híbrida, combinando features com camadas internas, por exemplo:

```
lib/  
  features/  
    login/  
      presentation/  
      domain/  
      data/
```

Por que essa estrutura é comum em projetos com Clean Architecture?

4. Questão-problema: um app acadêmico pequeno possui apenas três telas e não consome API. Qual estrutura de pastas seria mais adequada? Justifique.
5. Questão-problema: um app profissional possui login, produtos, carrinho, pagamento, notificações e perfil. Qual estrutura de pastas seria mais adequada? Justifique.
6. Compare as vantagens e desvantagens das três abordagens: por camada, por feature e híbrida.